Deep Learning



© 2023 Olivier Claerbout

Lesmateriaal

- Via GitHub account:
  git clone https://github.com/oclaerbout/syntra_data_scientist.git
- © 2023 Olivier Claerbout

  Dit materiaal mag niet worden gekopieerd, verspreid, gepubliceerd of anderszins gereproduceerd zonder uitdrukkelijke schriftelijke toestemming van de auteur. Dit geldt eveneens voor de Jupyter Notebooks!

Copyright Olivier Claerbout 2023

© 2023 Olivier Claerbout

Overzicht

- Echte vs Artificiële neurale netwerken
- Enkele voordelen van deep learning
- Tensors en tensor operaties
- Hoe werkt deep learning?
- Een eerste neuraal netwerk

© 2023 *Olivier Claerbout*

Copyright Olivier Claerbout 2024

Deep Learning

© 2023 *Olivier Claerbout*

Neurale netwerken

Neurale netwerken zijn een andere manier om te leren uit data

Neurale netwerken zijn geïnspireerd op de werking van onze hersenen

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Onze hersenen

- Onze hersenen bestaan uit cellen genaamd neuronen.
- Deze ontvangen, verwerken en versturen informatie.
- Ze zijn verbonden via synapsen waar informatie in de vorm van chemische en elektrische signalen wordt doorgegeven.
- Elk neuron kan verbonden zijn met duizenden andere neuronen, wat zorgt voor een complex netwerk. Wanneer een neuron informatie ontvangt die sterk genoeg is, 'vuurt' het en stuurt het signalen naar andere neuronen.
- Leren in de hersenen gebeurt door het versterken of verzwakken van deze synaptische verbindingen.

© 2023 Olivier Claerbout

Artificiële Neurale netwerken

- In een neuraal netwerk, hebben we ook 'neuronen' – eenvoudige, programmeerbare eenheden.
- Ze ontvangen input, verwerken deze, en geven een output.
- Ze zijn georganiseerd in lagen – inputlagen, verborgen lagen en outputlagen.
- Neuronen in een laag zijn verbonden met neuronen in de vorige en volgende laag. De 'sterkte' van deze verbindingen wordt bepaald door gewichten, vergelijkbaar met de sterkte van synaptische verbindingen in de hersenen.
- Net zoals de hersenen leren door synaptische verbindingen te versterken of te verzwakken, 'leert' een neuraal netwerk door de gewichten van zijn verbindingen aan te passen. Dit gebeurt tijdens een proces genaamd 'training', waarbij het netwerk voorbeelden krijgt en zijn gewichten aanpast om de gewenste output beter te benaderen.

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

De overeenkomsten

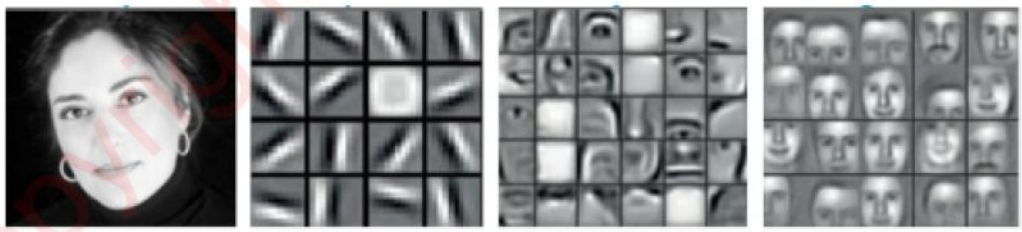
- De basisfunctionaliteit van een kunstmatig neuron en zijn verbindingen in een neuraal netwerk is geïnspireerd door hoe biologische neuronen en synapsen in de hersenen werken.
- Leren: Zowel de hersenen als kunstmatige neurale netwerken leren door de sterkte van hun verbindingen aan te passen, hoewel de mechanismen en de complexiteit aanzienlijk verschillen.
- Verwerking van Informatie: In beide systemen wordt informatie verwerkt in een netwerk van onderling verbonden eenheden (neuronen), wat leidt tot het vermogen om complexe taken uit te voeren, zoals herkennen van patronen, besluitvorming, en het leren uit ervaring.

© 2023 Olivier Claerbout

Deep Learning: enkele voordelen

© 2023 *Olivier Claerbout*

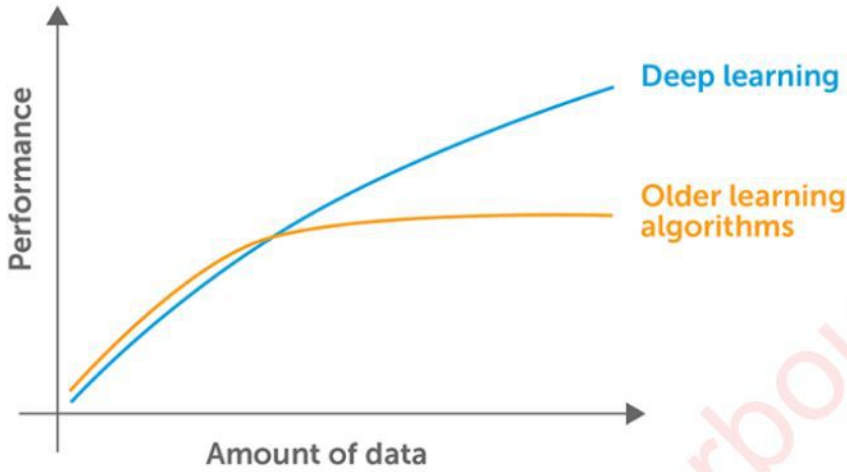
Voordelen van deep learning



© 2023 *Olivier Claerbout*

Copyright Olivier Claerbout 2023

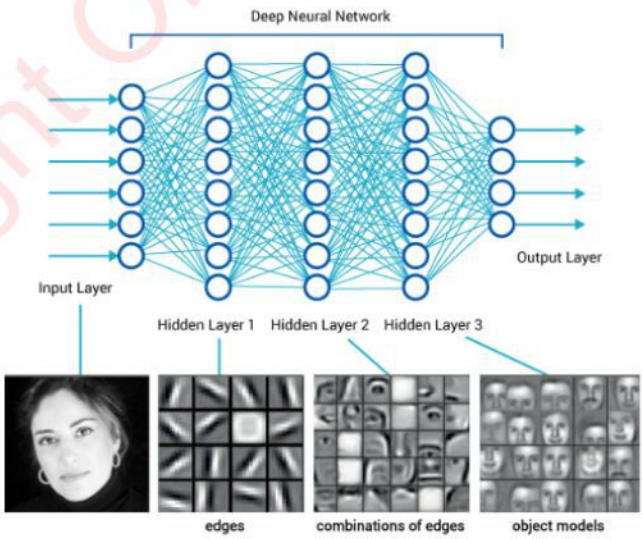
Voordelen van deep learning



© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Een neuraal netwerk



© 2023 Olivier Claerbout

Neurale netwerken - intro

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2023

Soorten neuralen netwerken

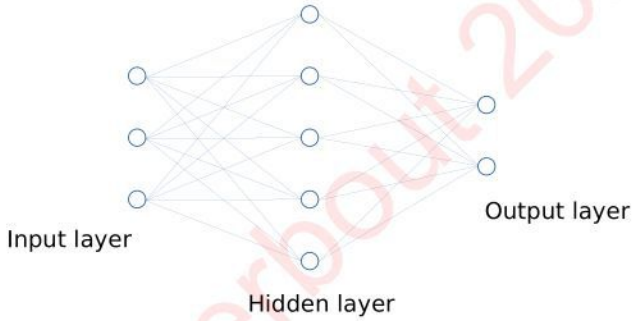
- Convolutional Neural Networks (CNN)
- Autoencoders (AE)
- Recurrent Neural Network (RNN)
- Long Short Term Memory (LSTM)
- Gated Recurrent Unit Net (GRU)
- Generative Adversarial Nets (GAN)
- Transformer Neural Nets (TNN)

Wij zien Vanilla Neural Nets

© 2023 Olivier Claerbout

Opbouw van een neuraal netwerk

- Neurale netwerken zijn grafen
- 1 of meerdere lagen
- Elke node in een laag is verbonden met elke node in de volgende laag
- Signalen gaan van input naar output via hidden layers (feedforward)
- In elk 'neuron' worden de inputs 'gewogen'
- Bij meerdere hidden layers spreken we van deep learning
- De output streeft naar een target

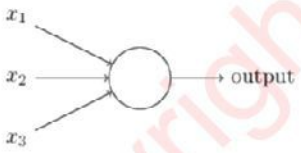


© 2023 Olivier Claerbout

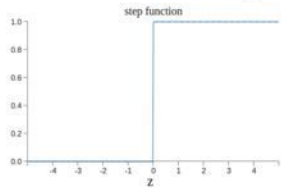
Copyright Olivier Claerbout 2024

Opbouw van een neuraal netwerk

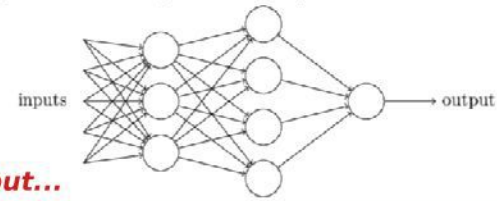
- Het Perceptron: een soort artificieel neuron
  - Een perceptron neemt verschillende binaire inputs, en produceert een enkele binaire output



$$output = \begin{cases} 0 \text{ if } \sum w_j x_j \leq treshold \\ 1 \text{ if } \sum w_j x_j > treshold \end{cases}$$



- De eerste layer 'weegt' de input
- De volgende layer 'weegt' de resultaten van de eerste layer. Dit is op een complexer en abstracter niveau dan in de eerste layer, enz...



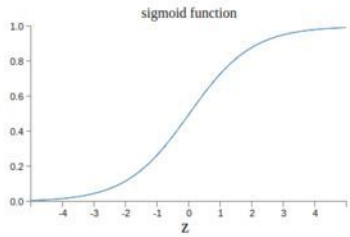
Enkel 0 & 1 als output...

© 2023 Olivier Claerbout

Opbouw van een neuraal netwerk

- Sigmoid neurons: aangepaste perceptrons zodat een klein verschil in w of b resulteert in een klein verschil in output (differentieerbaar)
- Deze kunnen een input tussen 0 en 1 accepteren, en de output is niet 0 of 1 maar :

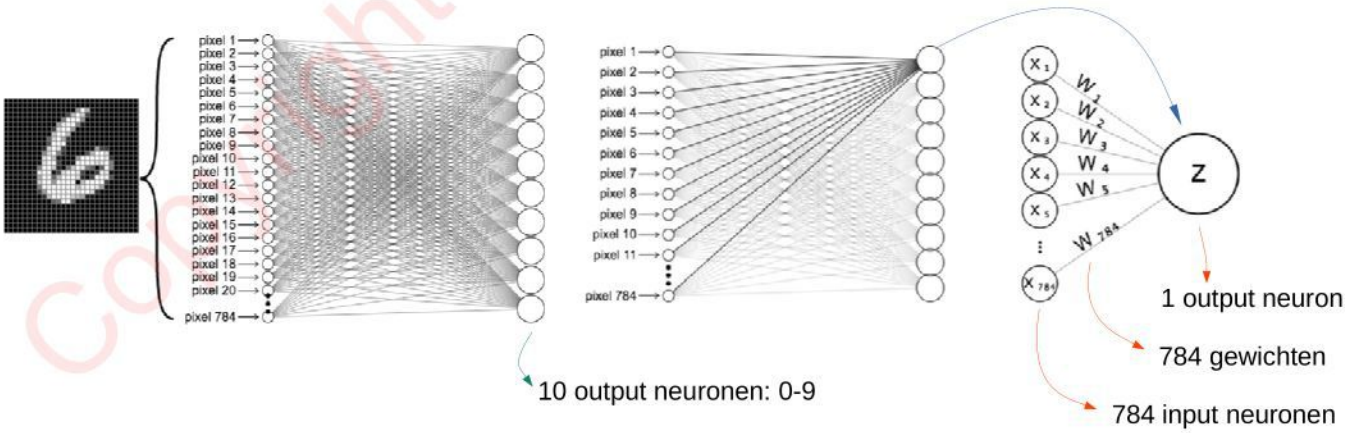
$$\sigma(z)=\frac{1}{1+e^{-z}}$$ *Sigmoid functie*



© 2023 Olivier Claerbout

Opbouw van een neuraal netwerk

- Voorbeeld: direct mappen van input layer aan output layer – geen hidden layers:



© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Opbouw van een neuraal netwerk

- Als we de weights die aankomen bij het eerste neuron zouden plotten als 28*28 grid:

$$\begin{bmatrix} X_1 & X_2 & X_3 & \dots & X_{28} \\ X_{29} & X_{30} & X_{31} & & X_{56} \\ X_{57} & X_{58} & X_{59} & & X_{84} \\ \vdots & & & \ddots & \\ X_{757} & X_{758} & X_{759} & & X_{784} \end{bmatrix} \circ \begin{bmatrix} W_1 & W_2 & W_3 & \dots & W_{28} \\ W_{29} & W_{30} & W_{31} & & W_{56} \\ W_{57} & W_{58} & W_{59} & & W_{84} \\ \vdots & & & \ddots & \\ W_{757} & W_{758} & W_{759} & & W_{784} \end{bmatrix} = Z$$

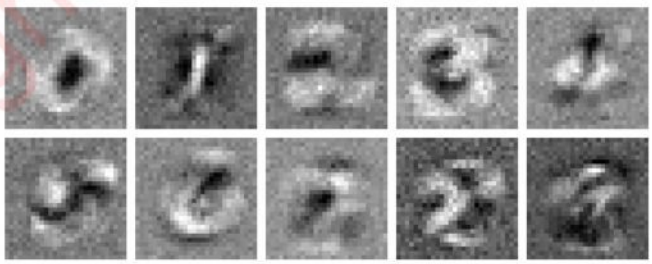
$$\begin{bmatrix} W_1 & W_2 & W_3 & \dots & W_{28} \\ W_{29} & W_{30} & W_{31} & & W_{56} \\ W_{57} & W_{58} & W_{59} & & W_{84} \\ \vdots & & & \ddots & \\ W_{757} & W_{758} & W_{759} & & W_{784} \end{bmatrix}$$



© 2023 Olivier Claerbout

Opbouw van een neuraal netwerk

- Alle weight matrices:



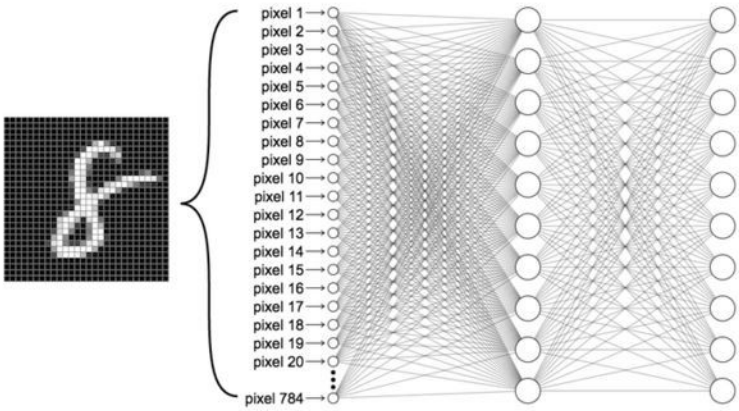
Een input image genereert een hoge activatie in een neuron als er veel overeenkomst is!

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Opbouw van een neuraal netwerk

- In dit model zonder hidden layers is elk gewicht slechts aan één class label gekoppeld. Het heeft dus geen impact op andere classes. Wat als we een hidden layer toevoegen?



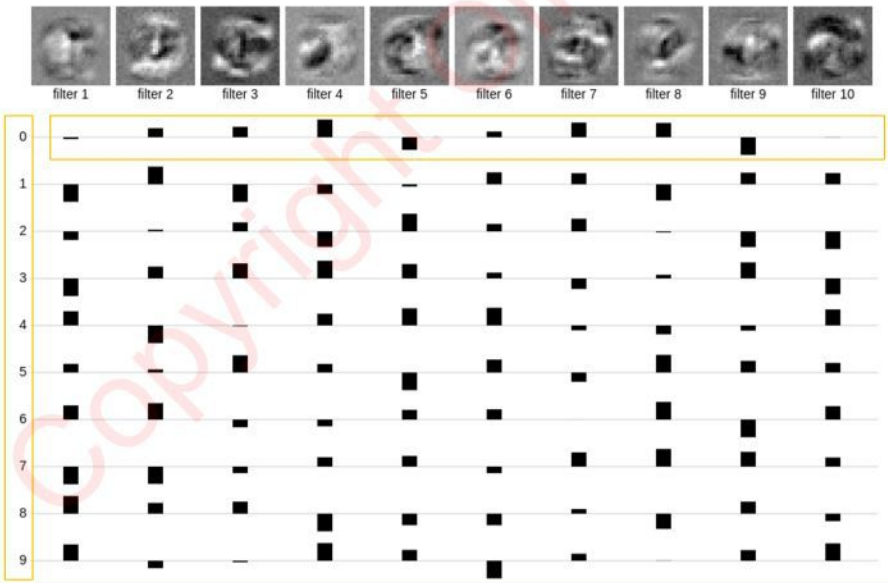
De weights in de hidden layer hebben nu invloed op elk van de output neuronen. De output is nu een gewogen combinatie van die 10 hidden activaties.

Hoe zien de weight matrices eruit in de hidden layer? En de weights naar de output neuronen?

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Binnen in de hidden layer...



→ output neuron met weight per hidden neuron

© 2023 Olivier Claerbout

Voordelen van hidden layers

- Flexibiliteit!
- Elke klasse heeft meer invoerpatronen die de overeenkomstige outputneuron stimuleren.
- Elke klasse kan worden geactiveerd door de aanwezigheid van (een combinatie van) verschillende abstracte kenmerken van de vorige verborgen laag
- We kunnen dus verschillende soorten nullen, verschillende soorten enen, enzovoort leren voor elke klasse.
- Dit zal meestal de prestaties van het netwerk voor de meeste taken verbeteren.

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Voor we verdergaan…
Wat zijn tensors?

© 2023 Olivier Claerbout

Tensors

Een tensor is een container voor numerieke data (dus voor getallen).

- rank 0 tensor: scalar (getal)
- rank 1 tensor: vector (array)
- rank 2 tensor: matrix
- rank 3 tensor (vb: timeseries: samples, timesteps, features)
- rank 4 tensor (images: samples, height, width, channels)

De eerste as is de samples as.
Deze wordt vaak de batch as genoemd.

© 2023 Olivier Claerbout

Tensors

- Belangrijkste eigenschappen van een tensor:
  - **Number of axes (rank):** *Vb: een rang-3 tensor heeft drie assen, en een matrix heeft twee assen. In Python bibliotheken zoals NumPy of TensorFlow noemt dit de ndim v/e tensor*
  - **Shape:** *Deze tuple beschrijft hoeveel dimensies de tensor heeft langs elke as.*
  - **Data type:** *(meestal dtype genoemd in Python bibliotheken) bijvoorbeeld float16, float32, float64, uint8, enz zijn. In TensorFlow kan je ook stringtensors tegenkomen.*

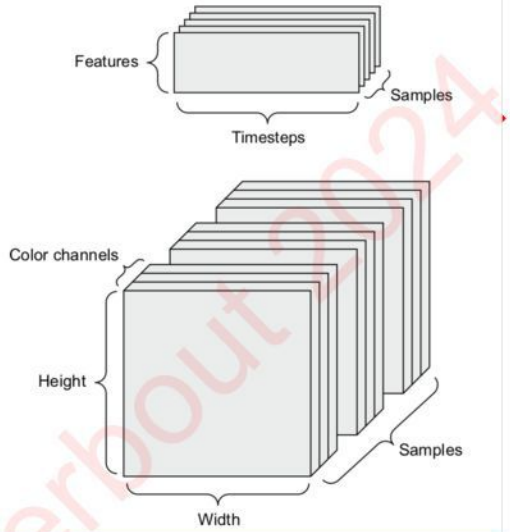
Copyright Olivier Claerbout 2024

© 2023 Olivier Claerbout

Tensors

- Enkele voorbeelden:
  - **Vector data:** *rang-2 tensors met als vorm (samples, features) waarbij elk sample een vector van numerieke attributen ('features') is.*
  - **Timeseries data:** *rang-3 tensors met als vorm (samples, timesteps, features) waarbij elk sample een sequentie (van lengte timesteps) van feature vectoren is.*
  - **Afbeeldingen:** *rang-4 tensors met als vorm (samples, frames, height, width, channels), waarbij elk sample een sequence (met lengte frames) van afbeeldingen is*

Wat gebeurt er met tensors in de neuronen?

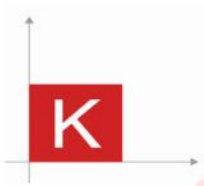


© 2023 Olivier Claerbout

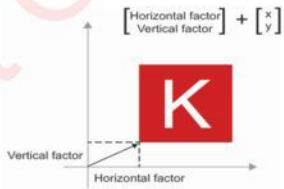
Copyright Olivier Claerbout 2024

Tensor operaties – geometrische interpretatie

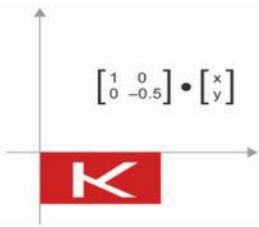
Basis



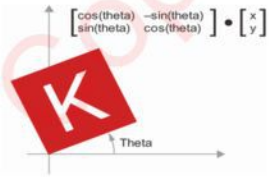
Translatie y = W + b



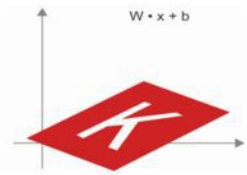
Schalen



Roteren



Affiene transformatie: vermenigvuldiging + translatie



$$y = W \cdot x + b$$

© 2023 Olivier Claerbout

Tensor operaties – geometrische interpretatie

Een neuraal netwerk is een ketting van allemaal tensor operaties die op zich eenvoudige geometrische transformaties zijn van de input data.

Je kan een neuraal netwerk beschouwen als een zeer complexe geometrische transformatie in een hoogdimensionale ruimte, opgebouwd uit een reeks eenvoudige stappen. Het komt overeen met het zoeken naar een representatie voor complex 'gevouwen' data in hoogdimensionale ruimtes. Elke laag in een neuraal netwerk voert een transformatie uit die de data opnieuw een beetje 'uit elkaar haalt'.



© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Werking van een neuraal netwerk

© 2023 Olivier Claerbout

Behind the scenes

- Wat gebeurt er in een neuraal netwerk?
  - Alles wat een deep neural net leert is terug te voeren tot een handvol tensor operaties die worden uitgevoerd op tensors
  - Een neuraal netwerk bestaat uit layers, bijvoorbeeld:

```
keras.layers.Dense(512, activation="relu")
```

  - Deze layer is te beschouwen als een functie die een input matrix bewerkt en als resultaat een nieuwe matrix teruggeeft - een ***nieuwe representatie van de input tensor***
  - Het aantal layers, het aantal neuronen in een layer,... dus de architectuur van het neuraal netwerk bepaalt de mogelijke representaties van de input tensor en dus de oplossingenruimte

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Behind the scenes

`keras.layers.Dense(512, activation="relu")` → **output = relu(dot(input, W) + b)**

- Deze 'functie' bestaat uit 3 tensor operaties:
  - Een dot product ('tensor product') van de input tensor en een tensor W
  - Bij dit resultaat wordt een vector b opgeteld°
  - Een activatie – vb een relu bewerking: relu(x) is max(x, 0) *(relu staat voor rectified linear unit – een activatiefunctie)*

°In neurale netwerken wordt voor activatie vaak een biasvector b toegevoegd aan de output van elk neuron. De biasvector biedt elk neuron een extra graad van vrijheid, waardoor het netwerk complexere patronen kan leren.

© 2023 Olivier Claerbout

Opbouw v/e NN

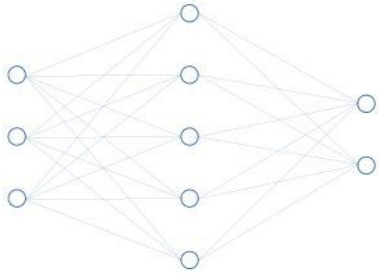
$h_i = ReLU(x \cdot w_i + b_i) = x_1 \times w_{11} + x_2 \times w_{w21} + x_3 \times w_{31} + b_1$

[1 × 3]

3 input features [x_1, x_2, x_3]

5 neuronen

2 output features



Weight matrix met shape (3,5) $\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{bmatrix}$

Bias vector met 5 elementen: $[b_1, b_2, b_3, b_4, b_5]$

Weight matrix met shape (5,2) $\begin{bmatrix} w'_{11} & w'_{12} \\ w'_{21} & w'_{22} \\ w'_{31} & w'_{32} \\ w'_{41} & w'_{42} \\ w'_{51} & w'_{52} \end{bmatrix}$

Bias vector met 2 elementen: $[b'_1, b'_2]$

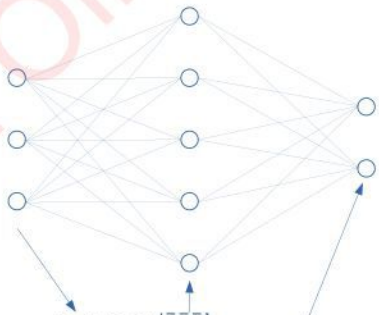
© 2023 Olivier Claerbout

Opbouw v/e NN

3 input features [x_1, x_2, x_3]

5 neuronen

2 output features



$[1\times3]\cdot[3\times5]=[1\times5]$

$[1\times5]\cdot[5\times2]=[1\times2]$

$\begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{bmatrix}$

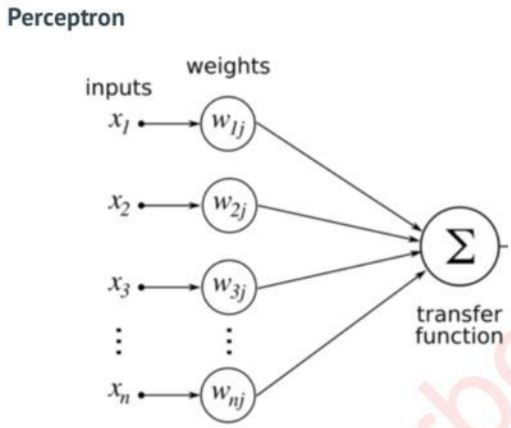
$\begin{bmatrix} w'_{11} & w'_{12} \\ w'_{21} & w'_{22} \\ w'_{31} & w'_{32} \\ w'_{41} & w'_{42} \\ w'_{51} & w'_{52} \end{bmatrix}$

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Waarom een activatiefunctie?

$$y = W \cdot x + b$$



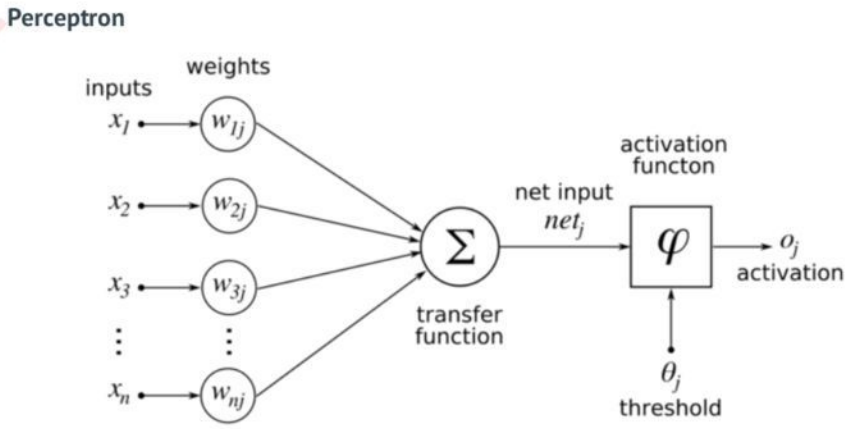
© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Waarom een activatiefunctie?

$$y = W \cdot x + b$$

Wat doen de weight matrix en bias vector?

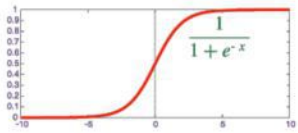


© 2023 Olivier Claerbout

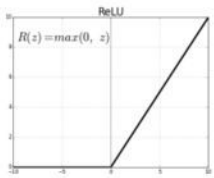
Activatie functies

- Enkele activatie functies:

Sigmoid



Relu = Rectified Linear Unit



Toevoegen niet-lineariteit om complexere patronen te kunnen leren!

© 2023 Olivier Claerbout

Deep ‘Learning’

Wat is “leren”?

- Zoeken beste “settings” voor alle weights en biases om de **loss functie te minimaliseren** bij een gegeven architectuur
- Aantal “settings”:
  - Input size vb 6 features
  - Aantal neuronen vb 10
  - Elk neuron is “connected” met elke feature: 6 * 10 = 60
  - Per neuron ook een bias: +10
  - Totaal: 70 “sliders”
  - Stel 2 lagen van 70 “sliders”: 4900 “sliders” ...



© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Deep ‘Learning’

Wat is “leren”?

- W en b zijn de trainbare parameters van de layer
- Ze bevatten de informatie die het model heeft geleerd door te worden blootgesteld aan training data
- Initieel bevatten de tensors W en b kleine random waarden
  - De initiële resulterende representaties (relu(dot(input, W))) zijn niet goed, maar vormen een startpunt
  - W en b worden aangepast op basis van een feedback signaal – training!

© 2023 Olivier Claerbout

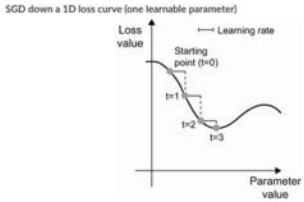
Copyright Olivier Claerbout 2024

Deep ‘Learning’

HOE leert het neurale netwerk?

- output = relu(dot(input, W) + b → W & b: trainbare parameters

1) Neem een aantal (een batch) samples, en de overeenkomende y_true
2) Run het model om een voorspelling voor y te doen – y_pred
3) Bereken hoe ver y_pred van y_true zit (loss function)
4) Bereken de gradient van de loss functie (backward pass)
5) Update de gewichten en biases ***“zo dat de loss functie volgende keer iets betere resultaten geeft”***



© 2023 Olivier Claerbout

Deep ‘Learning’

- *Update de gewichten en biases “zo dat de loss functie volgende keer iets betere resultaten geeft”* → ***Omhoog? Omlaag? Hoeveel?***
- Alle gebruikte functies (zoals dot of +) transformeren hun input op een continue manier
  - *Vb z = x + y : een kleine verandering in y resulteert in een kleine verandering in z*
- Uit de richting van de verandering in y volgt de richting van de verandering in z
- Wiskundig zou je zeggen dat deze functies differentieerbaar zijn.
- Als je dergelijke functies aan elkaar koppelt, is de grotere functie nog steeds differentieerbaar.

© 2023 Olivier Claerbout

Deep ‘Learning’

- We hebben een functie die de coëfficiënten van het model koppelt aan het loss van het model op een batch van gegevens
- Ook hiervoor geldt: *een kleine verandering in de coëfficiënten van het model resulteert in een kleine, voorspelbare verandering in de loss value.*
- Dit stelt je in staat om een wiskundige operator, genaamd de gradiënt (de afgeleide van een tensor operatie), te gebruiken om te beschrijven hoe het verlies varieert bij wijziging van de coëfficiënten van het model
- De berekende gradiënt kun je gebruiken om alle coëfficiënten tegelijk in één update in een richting te bewegen die het verlies vermindert.

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Deep ‘Learning’

- Het loss is minimaal als de afgeleide nul is
- Het analytisch zoeken van de combinatie van weights die de laagste loss functie geeft is praktisch niet mogelijk omwille van het aantal parameters
- Daarom wordt bijvoorbeeld mini-batch stochastic gradient descent gebruikt

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Mini Batch Stochastic° Gradient Descent

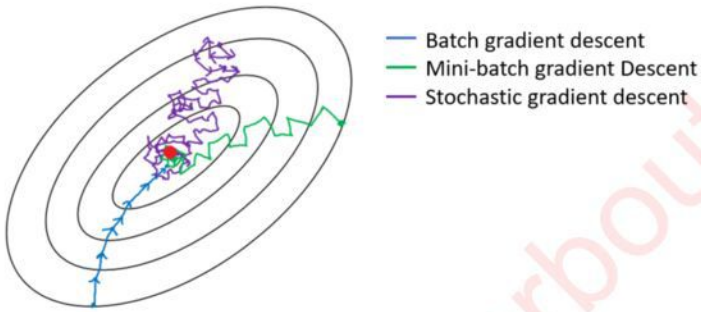
1)Kies een batch van trainingssamples x, en overeenkomstige doelwaarden, y_true
2)Run het model op x om voorspellingen te verkrijgen, y_pred (‘forward pass’)
3)Bereken het verlies van het model op de batch (= maatstaf voor de mismatch tssn y_pred en y_true)
4)Bereken de gradiënt van het verlies met betrekking tot de parameters van het model (‘backward pass’)
5)Verplaats de parameters een beetje in de tegenovergestelde richting van de gradiënt, bijvoorbeeld W -= learning_rate × gradient, waardoor het verlies op de batch een beetje vermindert. De leerfrequentie (hier learning_rate) is een scalaire factor die de "snelheid" van het gradiënt descent proces moduleert.

°Stochastic: elke batch wordt random geselecteerd

© 2023 Olivier Claerbout

Gradiënt Descent

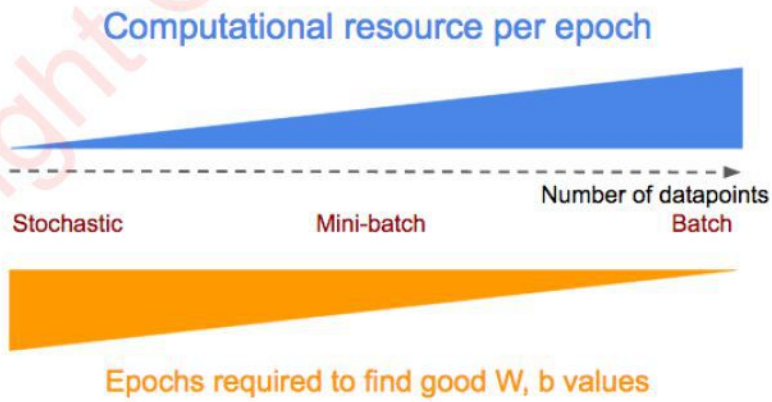
- Batch Gradient Descent = op alle data (Full Gradient Descent) – werkt niet voor grote datasets
- Mini Batch SGD = SGD op 1 < batch < aantal samples
- True SGD = op 1 voorbeeld (Stochastic Gradient Descent)



© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Benodigde Resources

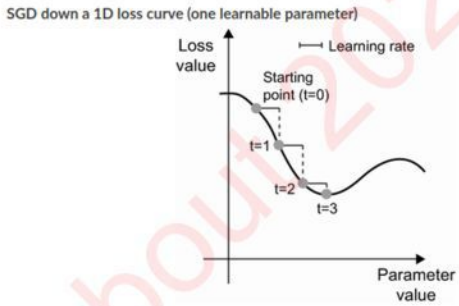


© 2023 Olivier Claerbout

Deep Learning

Problemen met Stochastic Gradient Descent

- Convergentiesnelheid
- Lokale minima



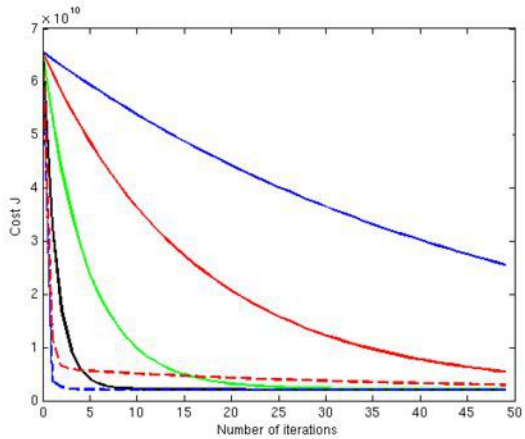
→ SGD met momentum, Adagrad, RMSprop,...: optimizers!

© 2023 Olivier Claerbout

Optimizers

Momentum & adaptive learning rates

- Versnellen van gradient descent
- Momentum



Hoe snel convergeert een model naar een lage cost value?

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Optimizers

Momentum & adaptive learning rates

- Adagrad: een adaptief optimalisatie-algoritme wat de learning rate voor elke parameter in het model (dus elk weight) individueel aanpast gebaseerd op hoeveel deze parameter reeds is aangepast = cache. Adagrad verlaagt de learning rate te agressief omdat de cache te snel groeit voor sommige parameters, waardoor het leren stopt.
- RMSprop: Adagrad variant: verminder de cache bij elke update:
  cache = decay x cache + (1-decay) x gradient^2
  - Decay is typisch 0,99 of 0,999
- Adam: meest gebruikte optimizer in deep learning. Combineert RMSprop met momentum.

Opgepast: experimenten tonen aan dat reguliere gradient descent een hogere performantie heeft dan adaptieve technieken.

© 2023 Olivier Claerbout

Copyright © Olivier Claerbout 2024

Hoe bouwen we een Neuraal netwerk?

© 2023 Olivier Claerbout

Opbouw van een neuraal netwerk

```
# Het model
model = Sequential()
model.add(Dense(16, input_dim=input_dim, kernel_initializer=initializer,activation=activation_hidden))
model.add(Dropout(dropoutrate))

# output layer
model.add(Dense(num_classes, kernel_initializer=initializer,activation=activation_output))

model.compile(loss='categorical_crossentropy',optimizer='RMSprop',metrics=['accuracy'])

# train the model

history = model.fit(X_train, y_train, epochs=epochs, validation_split = 0.2,verbose=1)
```


© 2023 Olivier Claerbout

Model Initialisatie

model = Sequential():

Hier initialiseer je een sequentieel model. Een sequentieel model in Keras is een lineaire stapel van lagen, waarbij elke laag precies één input tensor en één output tensor heeft.

Copyright Olivier Claerbout 2024

© 2023 Olivier Claerbout

Toevoegen van een layer

model.add(Dense(16, input_dim=input_dim, kernel_initializer=initializer, activation=activation_hidden))

Hier voeg je een 'Dense' laag toe aan het model. Een 'Dense' laag is een volledig verbonden (fully connected) neuronen laag.

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Toevoegen van een layer

De parameters van de dense laag zijn:

- 16: Het aantal neuronen in de laag.
- input_dim: De grootte van de inputlaag (hoeveel inputfeatures er zijn).
- kernel_initializer: De methode die wordt gebruikt voor het initialiseren van de gewichten.
  - Random Normal: kleine willekeurige waarden uit een normale verdeling
  - Random Uniform; willekeurige kleine waarden uit een gelijkmatige verdeling
  - ...

© 2023 Olivier Claerbout

Toevoegen van een layer

Initialisatie van de weights is belangrijk:

- Constanten leiden tot symmetrieën – te vermijden!
- Bias mag op 0 geïnitialiseerd worden
- Verschillende random methodes beschikbaar

© 2023 Olivier Claerbout

Toevoegen van een layer

Dense layer parameters - activation (https://keras.io/api/layers/activations/)

- relu: Een veelgebruikte activatiefunctie die de waarde 0 uitvoert voor negatieve inputs en de waarde onveranderd laat voor positieve inputs. Helpt bij het verminderen van het probleem van vanishing gradiënts. Zeer efficiënt naar rekenkracht; veel activaties worden 0. Nadeel: dode neuronen kunnen niet meer geactiveerd worden. Gebruik voor hidden layers.
- sigmoid: Geeft een waarde tussen 0 en 1 terug, wat handig is voor uitvoerlagen in *binaire classificatieproblemen*. Enkel nog als output layer bij binaire classificatie. Vanishing gradient!
- softmax: Geeft een reeks waarden die de waarschijnlijkheden representeren die overeenkomen met elke klasse in een *multiclass classificatieprobleem*. De waarden sommeren op tot 1.
- Leaky ReLU: een variant van ReLU die een kleine, niet-nul gradiënt toestaat wanneer de unit niet actief is. Geen dode neuronen; wel meer parameters om te trainen.
- ...

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Toevoegen van een layer

Dense layer parameters – activation conclusie:

- Voor hidden layers:
  - Gebruik eerst ReLu
  - Probeer Leaky Relu
  - Geen Sigmoid of Tanh!
- Voor output layer:
  - Lineair bij regressie
  - Softmax / Sigmoid bij classificatie

© 2023 Olivier Claerbout

Toevoegen van een layer

Wat is het Vanishing gradient probleem?

Het vanishing gradient probleem is een obstakel bij het trainen van diepe neurale netwerken, waarbij de gradiënten die gebruikt worden om de gewichten aan te passen zo klein worden (vooral Sigmoid of Tanh kunnen gradiënten met heel kleine waarden hebben) dat de vroegere lagen in het netwerk nauwelijks nog bijgewerkt worden of leren. Dit effect is sterker bij een groter aantal layers door het herhaaldelijk vermenigvuldigen van kleine gradiënten tijdens backpropagation. Het kan worden aangepakt door specifieke activatiefuncties zoals ReLU te gebruiken, gewichten zorgvuldig te initialiseren, normalisatietechnieken toe te passen of netwerkarchitecturen aan te passen.*

**Wanneer je een diep neuraal netwerk traint, wordt de fout (of het verlies) van het netwerk teruggevoerd van de outputlaag naar de inputlaag, zodat de gewichten aangepast kunnen worden om de prestatie te verbeteren.*

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Toevoegen van een layer

De parameters van de dense laag zijn:

- **model.add(Dropout(dropoutrate))**: Dropout is een reguleringstechniek om overfitting te voorkomen door willekeurig een deel van de neuronen 'uit te schakelen' tijdens het trainingsproces. De dropoutrate bepaalt het percentage van neuronen dat in elke updatecyclus wordt genegeerd.
- **model.add(BatchNormalization())**:Normaliseert de inkomende data in elke laag. Dit zorgt dat elke laag iets meer onafhankelijk leert van de andere lagen, reduceert overfitting door een licht regulariserend effect, en voegt een beetje ruis toe aan elke layer waardoor minder dropout nodig is.

Input data → Batch Normalization → Activatie Functie → volgende layer

© 2023 Olivier Claerbout

Output laag

model.add(Dense(num_classes, kernel_initializer=initializer, activation=activation_output))

- Een tweede 'Dense' laag wordt toegevoegd die de *outputlaag* van het model vormt. De num_classes parameter geeft *het aantal output neuronen aan, wat overeenkomt met het aantal klassen in een classificatieprobleem*.

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Compileren van het model

model.compile(...): Met deze functie compileer je het model door de loss functie, optimizer en metrics te definiëren die gebruikt zullen worden voor het trainen.

- loss='categorical_crossentropy': De loss functie die gebruikt wordt voor multiclass classificatieproblemen.
- optimizer='RMSprop': optimalisatiealgoritme wat gebruikt wordt om de gewichten aan te passen.
- metrics=['accuracy']: De lijst van metrics die geëvalueerd worden door het model tijdens training en testen.

© 2023 Olivier Claerbout

Trainen van het model

history = model.fit(...)

- X_train en y_train: *De trainingsdata en labels.*
  - *Bij tekstlabels kan je deze via LabelEncoder omzetten naar een numeriek formaat*
  - *Categorische labels dienen one-hot encoded te worden*
- batch_size: *optioneel; aantal samples per gradiënt update. Typisch tussen 8 en 128. Deze samples worden simultaan verwerkt. Meestel 2^n omwille van allocatie van geheugen in de GPU.*
- epochs: *Het aantal iteraties over de gehele dataset voor het trainen*
- validation_split: *Het % data dat gebruikt zal worden voor validatie.*
- verbose=1: *Geeft aan dat je de trainingsvoortgang wilt zien*
- *Het history object dat geretourneerd wordt bevat de trainingsgeschiedenis, inclusief de trainings- en validatieloss en nauwkeurigheid voor elk epoch. Dit kan gebruikt worden voor het visualiseren van de trainingsvoortgang en het diagnosticeren van problemen met het trainen, zoals over- of underfitting*

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Grootte van een neuraal netwerk

© 2023 Olivier Claerbout

Opbouw van een neuraal netwerk

- Het aantal verborgen lagen (hidden layers) en het aantal neuronen per laag in een neuraal netwerk definieert de oplossingenruimte.
- Er zijn geen vaste regels, maar wel richtlijnen en methoden die je kunnen helpen bij het maken van deze keuzes.
- De keuze hangt af van:
  - de complexiteit van de taak,
  - de hoeveelheid en soort data,
  - de gewenste nauwkeurigheid van het model

Copyright Olivier Claerbout 2023

© 2023 Olivier Claerbout

Opbouw van een neuraal netwerk

- Aantal Verborgen Lagen:
  - Voor veel eenvoudige problemen kan één verborgen laag voldoende zijn. Neurale netwerken met één verborgen laag kunnen al een verrassend groot aantal functies en besluitvormingsprocessen modelleren.
  - Voor complexere problemen, zoals beeld- of spraakherkenning, zijn meerdere verborgen lagen (deep learning) nodig. Deze extra lagen helpen het netwerk om complexere patronen en relaties in de data te leren.

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Opbouw van een neuraal netwerk

- Aantal Neuronen per Laag:
  - Input- en Outputlagen: Het aantal neuronen in de inputlaag komt overeen met het aantal features (kenmerken) van je data. Het aantal neuronen in de outputlaag komt overeen met het aantal outputs dat je model moet produceren.
  - Verborgen Lagen: Dit is meer een 'kunst' dan een exacte wetenschap. Een algemene richtlijn is om te beginnen met een aantal dat ligt tussen het aantal input- en outputneuronen. Te weinig neuronen kunnen leiden tot underfitting (het model is te eenvoudig), terwijl te veel neuronen kunnen leiden tot overfitting (het model is te complex en generaliseert slecht).
  - *Begin met een schatting en pas het aantal neuronen aan op basis van de prestaties van het model. Gebruik technieken zoals kruisvalidatie om te bepalen of je model goed presteert.*

© 2023 Olivier Claerbout

Opbouw van een neuraal netwerk

- Andere Overwegingen:
  - Complexiteit van de Taak: Meer complexe taken vereisen over het algemeen diepere netwerken met meer neuronen.
  - Hoeveelheid en Soort Data: Meer data en meer gevarieerde data kunnen complexere modellen rechtvaardigen.
  - Rekenkracht en Trainingstijd: Meer neuronen en lagen vergen meer rekenkracht en langere trainingstijd. Zorg dat je de capaciteiten van je hardware in overweging neemt.

© 2023 Olivier Claerbout

Opbouw van een neuraal netwerk

Conclusie:

Het bepalen van het aantal verborgen lagen en neuronen is grotendeels een proces van experimenteren en finetunen.

Het is belangrijk om met een redelijke schatting te beginnen en vervolgens het model te testen en aan te passen op basis van de resultaten.

Het gebruik van geavanceerde technieken zoals automatische hyperparameteroptimalisatie (bijvoorbeeld via grid search of random search) kan ook helpen bij het vinden van de optimale configuratie.

© 2023 Olivier Claerbout

Copyright Olivier Claerbout 2024

Neurale netwerken in praktijk!

© 2023 Olivier Claerbout

SYNTRA ANTWERPEN & VLAAMS-BRABANT

Copyright Olivier Claerbout 2023